

5

ENCODING ENGLISH CIVIL WAR CIPHERS



In 1587, Mary, Queen of Scots, lost her head due to a scrap of paper. Fifty-five years later, Sir John Trevanion, a supporter of another beheaded monarch, Charles the First, had his head saved by a scrap of paper. What made the difference? Steganography.

Steganography is the time-tested practice of concealing messages so well that their existence isn't even suspected. The name is based on the Greek words for "covered writing," and a very literal Grecian example was to take wax-covered wooden tablets used for writing, scrape off the wax, write on the wood, and then cover the board with a new coating of smooth wax. A modern-day example is to embed a message in an image by subtly altering its color components. Even a simple 8-bit JPEG image contains more colors than the human eye can detect, so without digital processing or filtering, the message is essentially invisible.

In this chapter, you'll work with the *null cipher*, which isn't a cipher at all but a steganographic technique for concealing plaintext within other strings of noncipher material. *Null* means "none," so with a null cipher, you have chosen not to encrypt the message. The following is an example of a null cipher using the first letter in every word:

Nice **u**ncles **l**ive **l**onger. **C**ruel, **i**nsensitive **p**eople **h**ave **e**ternal **r**egrets.

First, you'll write code that finds the hidden message that saved Sir John, and then you'll accomplish the much more difficult task of writing a null cipher. Finally, you'll get the opportunity to write a program that might have saved Mary's head, had she used the output.

Project #10: The Trevanion Cipher

Queen Mary relied on both steganography and encryption to protect her messages. The strategy was sound, but her application was flawed. Unknowingly, she relied on a double agent named Gilbert Gifford to smuggle her messages. Gifford first delivered them to Queen Elizabeth's spymaster, who cracked the cipher and replaced it with a forged message that enticed Mary to incriminate herself. The rest, as they say, is history.

For John Trevanion, the outcome was rosier. Sir John, a distinguished cavalier who aided Charles I against Oliver Cromwell in the English Civil War, was captured and imprisoned in Colchester Castle. The day before his execution, he received a letter from one of his friends. The letter was not smuggled in but delivered straight into the hands of his jailors, who examined it but didn't notice any deception. After reading it, Sir John asked for some time alone to pray in the chapel. When his jailors came back to fetch him, he had vanished.

Here is the message Sir John received:

Worthie Sir John: Hope, that is the beste comfort of the afflicted, cannot much, I fear me, help you now. That I would saye to you, is this only: if ever I may be able to requite that I do owe you, stand not upon asking me. 'Tis not much I can do: but what I can do, bee you verie sure I wille. I knowe that, if deathe comes, if ordinary men fear it, it frights not you, accounting for it for a high honour, to have such a rewarde of your loyalty. Pray yet that you may be spared this soe bitter, cup. I fear not that you will grudge any sufferings; onlie if bie submission you can turn them away, 'tis the part of a wise man. Tell me, an if you can, to do for you anythinge that you wolde have done. The general goes back on Wednesday. Restinge your servant to command. R.T.

As you have probably guessed, this seemingly innocent letter contains a hidden message, revealed below in bold:

Worthie Sir John: **H**ope, **t**hat is the beste comfort of the afflicted, **c**annot much, I **f**ear me, **h**elp you now. **T**hat I would saye to you, is **t**his only: if **e**ver I may be able to requite that I do owe you, **s**tand not upon asking me. 'Tis not much I can do: **b**ut what I can do, **b**ee you verie sure I wille. I **k**nowe that, if **d**eathe comes, if **o**rdinary men fear it, it **f**righs not you, **a**ccounting for it for a high honour, to **h**ave such a rewarde of your loyalty. **P**ray yet that you may be spared this soe bitter, cup. I **f**ear not that you will grudge any sufferings; **o**nlie if bie submission you can turn them away, 'tis the part of a wise man. **T**ell me, an **i**f you can, to **d**o for you any-thinge that you wolde have done. **T**he general goes back on Wednesday. **R**estinge your servant to command. R.T.

This null cipher uses every third letter after a punctuation mark to let Sir John know that a “panel at east end of chapel slides.” It’s rumored that the remains of a narrow stairway were later discovered in a recess in a wall in the castle. The passage was blocked at the time of discovery, but it may have been Sir John’s escape route around 1642.

This last-minute escape would not have been possible with a traditional cipher. Only by expertly concealing the message with steganography was its author able to get it so quickly into Sir John’s hands. And the beauty of a null cipher is that, even if Sir John didn’t know the pattern but suspected a message was present, he could have found it fairly quickly.

If Sir John’s friend had been more careful, concealing encrypted cipher-text instead of plaintext, Sir John probably wouldn’t have deciphered the message in the short time he had remaining—unless he’d been informed beforehand of the cipher type and key.

THE OBJECTIVE

Write code that finds the letters hidden after punctuation marks in a null cipher and lets the user choose the number of letters after a punctuation mark to search for a solution.

Strategy and Pseudocode

Null ciphers rely on a repeating pattern known to both the sender and receiver. For example, every third word may be part of the real message or, better, the last letter of every third word. In the Trevanion cipher, it's the third letter after a punctuation mark.

To find the Trevanion cipher, assume punctuation marks are the signal to begin counting, and then write code that locates every n th letter after the mark and saves those letters to a string or list. Once you've worked out how to do this, you can easily edit the code to work with any starting point, such as every capitalized word, every second letter per word, or the starting letter of every third word.

The only real point of contention involves punctuation marks. For instance, did the null cipher's writer *want* punctuation marks to be included in the plaintext? How do you handle a second punctuation mark within the desired count range? What happens if two punctuation marks occur in succession?

If you take a close look at the Trevanion cipher, you should see that there are double punctuations caused by the repeated use of the word *'tis*. There is also a jumble of punctuation marks at the end of the message, where the writer provides his initials. To deal with this, Sir John and his friend may have established some rules before Sir John's incarceration, or Sir John just used trial and error to work them out.

Based on the end of the message, punctuation marks aren't included in the letter count. If Sir John's friend had intended them to be, then the hidden message would end with a capital *T*, because the *T* is three *characters* after a punctuation mark, and crucially not three *letters* after. This means that, if the reader encounters a punctuation mark within the count limit, they have to restart their count.

So these are the rules:

- Initiate a letter count with every punctuation mark.
- Reset the count if a punctuation mark is encountered.
- Punctuation marks cannot be part of the plaintext message.

Since you may not know what the letter count should be, write the code so that it checks all counts up through a limit the user provides. The pseudocode is fairly straightforward:

```
Load a text file and strip it of whitespace
Get user input on how many letters after punctuation to look ahead and examine
Loop through number of letters from 1 to this lookahead value
    Start an empty string to hold the translation
    Start a counter
    Start a ❶first-found marker and set to False
    Loop through characters in the text
        If character is punctuation
            Counter = 0
            First-found = True
        Otherwise, if ❷first-found is True
            Counter + 1
        If counter = lookahead value
            Add character to translation string
    Display translation for this lookahead value
```

Note that the first-found variable ❶ will remain `False` until a punctuation mark is encountered, after which it will be set to `True` ❷. This prevents the program from counting until the first punctuation mark is found.

Now you're ready to write the code!

The Trevanion Cipher Code

The code in this section will find a Trevanion-type null cipher encoded with a specific number of letters after each punctuation mark. You will also need the text file containing the Trevanion cipher. You can download both the script and the text file from <https://www.nostarch.com/impracticalpython/> as `null_cipher_finder.py` and `trevanion.txt`, respectively. Keep these files in the same folder.

Loading the Text

Listing 5-1 imports some useful modules and loads the text file containing the null cipher.

null_cipher_finder.py, part 1

```
❶ import sys
    import string

❷ def load_text(file):
    """Load a text file as a string."""
    ❸ with open(file) as f:
        ❹ return f.read().strip()
```

Listing 5-1: Imports modules and loads the null cipher text

First, import the now-familiar `sys` module so you can handle exceptions that may occur during user input ❶. Also import the `string` module to gain access to useful collections of constants, like letters and punctuation marks.

Next, define a function to load the text file containing the null cipher ❷. This function is similar to the one you used to load a dictionary file in **Chapter 2**. It will be called by the `main()` function later to actually load the file.

Start the `load_text()` function by using `with` to open the file ❸. By using `with`, you know the file will be automatically closed after it is loaded. Use `read()` to load the contents and `strip()` to remove leading and trailing whitespace. Note that you can do this on the same line with the `return` statement ❹.

Finding the Hidden Message

Listing 5-2 defines the function that finds the hidden message. It takes two arguments. The first is the message, which is the original text file as a string stripped of whitespace, and the second is the number of letters to check after a punctuation mark. This check value is obtained from the user as part of the `main()` function.

```

def solve_null_cipher(message, lookahead):
    """Solve a null cipher based on number of letters after punctuation mark.

    message = null cipher text as string stripped of whitespace
    lookahead = endpoint of range of letters after punctuation mark to examine
    """
    ❶ for i in range(1, lookahead + 1):
        ❷ plaintext = ''
        count = 0
        found_first = False
        ❸ for char in message:
            ❹ if char in string.punctuation:
                count = 0
                found_first = True
            ❺ elif found_first is True:
                count += 1
            ❻ if count == i:
                plaintext += char
        ❼ print("Using offset of {} after punctuation = {}".
              format(i, plaintext))
        print()

```

Listing 5-2: Searches for hidden letters

Treat the `lookahead` value as the endpoint of a range in a `for` loop so that you can check all the intervening letters in the message for the presence of a hidden message. Set the range as `(1, lookahead + 1)` ❶; that way, you start with the first letter after a punctuation mark and include the user's choice in the evaluation.

Now, assign a few variables ❷. First, initialize an empty string to hold the translated plaintext. Then set a counter to 0. Finally, set a `found_first` variable to `False`. Remember that the program uses this variable to defer counting until the first punctuation mark is encountered.

Next, begin looping through the characters in the message ❸. If you encounter a punctuation mark, reset the counter to 0 and set `found_first` to `True` ❹. If you've found a punctuation mark already and the current character isn't punctuation, advance the counter by 1 ❺. If you've found the letter you're looking for—meaning the count has reached the current `lookahead` value (`i`)—add the letter to the plaintext string ❻.

When you've examined all the characters in the message for the current `lookahead` value, display the current key and the translation ❼.

Defining the `main()` Function

Listing 5-3 defines the `main()` function. You may remember from **Chapter 3** that the `main()` function is like your program's project manager: it takes input, keeps track of progress, and tells the other functions when to work.

null_cipher_finder.py, part 3

```
def main():
    """Load text, solve null cipher."""
    # load & process message:
    ❶ filename = input("\nEnter full filename for message to translate: ")
    ❷ try:
        loaded_message = load_text(filename)
    except IOError as e:
        print("{}\nTerminating program.".format(e), file=sys.stderr)
        sys.exit(1)
    ❸ print("\nORIGINAL MESSAGE =")
    print("{}\n".format(loaded_message), "\n")
    print("\nList of punctuation marks to check = {}".format(string.punctuation), "\n")

    # remove whitespace:
    ❹ message = ''.join(loaded_message.split())

    # get range of possible cipher keys from user:
    ❺ while True:
        ❻ lookahead = input("\nNumber of letters to check after " \
```



```

        "punctuation mark: ")
    ⑦ if lookahead.isdigit():
        lookahead = int(lookahead)
        break
    else:
    ⑧ print("Please input a number.", file=sys.stderr)
print()

# run function to decode cipher
    ⑨ solve_null_cipher(message, lookahead)

```

Listing 5-3: Defines the `main()` function

Start by asking the user for the name of the file (name + extension) ❶, and then use `try` to call the `load_text()` function ❷. If the file can't be found, print the error in red—for those using the IDLE window—and exit the program using `sys.exit(1)`, where the `1` indicates termination with an error.

Print the message followed by the list of punctuation marks in the `string` module ❸. Only the characters in this list will be recognized by the program as punctuation.

Next, take the loaded message and remove all spaces ❹. You're going to count only letters and punctuation marks, so spaces would just get in the way. Start a `while` loop that keeps asking the user for input in the event they enter a bad value ❺. Ask the user for the number of letters to check after a punctuation mark ❻. This will be treated as a range, starting with `1` and ending with the user's choice plus `1`. If the input value is a digit ❼, turn it into an integer, since `input` returns a string. Then, use `break` to exit the loop.

If the user enters an invalid value, like “Bob,” use a `print` statement to request a number and, for shell users, make the font red using `sys.stderr` ❽. The `while` loop will then repeat the request for input.

Pass the `lookahead` variable, along with the `message`, to the `solve_null_cipher` function ❾. Now all that's left is to call the `main()` function.

Running the main() Function

Back in the global space, complete the code by calling `main()`—but only if the program is run in stand-alone mode versus being imported into another program (**Listing 5-4**).

null_cipher_finder.py, part 4

```
if __name__ == '__main__':  
    main()
```

Listing 5-4: Calls the `main()` function

The following is example output from the completed program, using the Trevanion cipher as input:

```
Enter full filename for message to translate: trevanion.txt
```

```
ORIGINAL MESSAGE =
```

```
Worthie Sir John: Hope, that is the beste comfort of the afflicted, cannot  
much, I fear me, help you now. That I would saye to you, is this only: if ever  
I may be able to requite that I do owe you, stand not upon asking me. 'Tis not  
much I can do: but what I can do, bee you verie sure I wille. I knowe that,  
if deathe comes, if ordinary men fear it, it frights not you, accounting for  
it for a high honour, to have such a rewarde of your loyalty. Pray yet that  
you may be spared this soe bitter, cup. I fear not that you will grudge any  
sufferings; onlie if bie submission you can turn them away, 'tis the part of a  
wise man. Tell me, an if you can, to do for you anythinge that you wolde have  
done. The general goes back on Wednesday. Restinge your servant to command.  
R.T.
```

```
List of punctuation marks to check = !"#$%&'()*+,-./:;<=>?@[\\]^_`{|}~
```

```
Number of letters to check after punctuation mark: 4
```

Using offset of 1 after punctuation = HtcIhTiisTbbIiiiatPcIotTatTRRT

Using offset of 2 after punctuation = ohafehsftiuekffftcorufnienohe

Using offset of 3 after punctuation = panelateastendofchapelslides

Using offset of 4 after punctuation = etnaphvnnwyoerroayaitlfogt

In this output, the program has checked up to the fourth letter after a punctuation mark, but as you can see, it finds the solution using three letters after a punctuation mark.

Project #11: Writing a Null Cipher

Here is an unfinished example of a very weak null cipher based on the start of each word. Take a minute and try to complete the sentence:

H_____ e_____ l_____ p_____ m_____ e_____.

You probably found it difficult, because whether you use letters or even whole words, it takes hard work and time to produce a null cipher that doesn't read awkwardly and arouse suspicion. The heart of the problem is context. If the cipher is encapsulated within correspondence, that correspondence has to be coherent to avoid suspicion. That means it has to address a relevant topic and stay true to that topic for a reasonable number of sentences. As you probably saw, drafting even one sentence on any topic is no easy task!

The key is to credibly avoid context, and a good way to do this is with a list. No one expects a shopping list to be rigidly organized or make sense. Lists can also be tailored to the receiver. For example, correspondents might get into a discussion of books or movies and exchange lists of their favorites. A prisoner might start studying a foreign language and receive regular vocabulary lists from their tutor. A businessperson might get monthly inventories from one of their warehouses. With lists, context is honored even while words are shuffled so the correct letter is found in the correct place.

THE OBJECTIVE

Write code that hides a null cipher within a list of words.

The List Cipher Code

The *list_cipher.py* code, in **Listing 5-5**, embeds a null cipher within a list of dictionary words under the deception of vocabulary training. You'll also need the *load_dictionary.py* program you used in **Chapters 2** and **3**. You can download this file, along with the following script, from <https://www.nostarch.com/impracticalpython/>. Finally, you'll need one of the dictionary files you used in **Chapters 2** and **3**. You can find a list of suitable online dictionaries in **Table 2-1** on **page 20**. All of the aforementioned files should be kept in the same folder.

list_cipher.py

```
❶ from random import randint
    import string
    import load_dictionary

    # write a short message that doesn't contain punctuation or numbers!
    input_message = "Panel at east end of chapel slides"

    message = ''
    for char in input_message:
        ❷ if char in string.ascii_letters:
            message += char
    print(message, "\n")
    ❸ message = "".join(message.split())

    ❹ # open dictionary file
    word_list = load_dictionary.load('2of4brif.txt')

    # build vocabulary word list with hidden message
    ❺ vocab_list = []
```

```

❹ for letter in message:
    size = randint(6, 10)
    ❶ for word in word_list:
        if len(word) == size and word[2].lower() == letter.lower()\
        and word not in vocab_list:
            vocab_list.append(word)
            break

❸ if len(vocab_list) < len(message):
    print("Word List is too small. Try larger dictionary or shorter message!")
else:
    print("Vocabulary words for Unit 1: \n", *vocab_list, sep="\n")

```

Listing 5-5: Hides null cipher in a list

Start by importing the `random` module's `randint()` function ❶. This permits the (pseudo)random selection of an integer value. Then load the `string` module, for access to ASCII letters. Finish by importing your `load_dictionary` module.

Next, write a short secret message. Note that the associated comment forbids punctuation marks or numbers. Trying to use these with a dictionary file's contents would be problematic. So, filter out everything but letters by checking for membership in `string.ascii_letters`, which contains both uppercase and lowercase letters ❷:

```
'abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ'
```

Display the message and then remove the whitespace ❸. Load your dictionary file ❹ and start an empty list to hold the vocabulary words ❺.

Use a `for` loop to go through each letter in the message ❻. Name a `size` variable and assign it a random value between 6 and 10 using the `randint()` function. This variable will ensure the words are long enough to be credible as vocabulary words. You can set the maximum value higher if you wish.

Nest another `for` loop and use it to go through the dictionary words ❷, checking their length against the `size` variable and comparing the (lowercase) letter at index 2—the word’s third letter—to the current (lowercase) letter in the message loop. You can change the index value on the word, but make sure it doesn’t exceed the lowest possible `size` variable minus 1! A final comparison prevents the same word from being used twice. If the word passes the tests, append it to `vocab_list` and move on to the next letter in the message.

A typical dictionary file should contain enough words to encrypt a short message. But, to be safe, use a conditional to check that the length of `vocab_list` is not shorter than the length of the message ❸. If it’s shorter, then you ran out of words before reaching the end of the message, and you need to print a warning for the user. Otherwise, print the list of words.

The List Cipher Output

Here is the output from the code (I’ve highlighted every third letter for readability, though the message is pretty easy to spot without any aid):

```
Panelateastendofchapelslides
```

```
Vocabulary words for Unit 1:
```

```
alphabets  
abandoning  
annals  
aberration  
ablaze  
abandoned  
acting  
abetted  
abasement  
abseil  
activated  
adequately  
abnormal
```

abdomen
abolish
affecting
acceding
abhors
abalone
ampersands
acetylene
allegation
absconds
aileron
acidifying
abdicating
adepts
absent

Using a font with a consistent character width and stacking the words really compromises the cipher. We'll look at ways to deal with this in **"Saving Mary"** on **page 102**.

Summary

In this chapter, you wrote a program that reveals the hidden message in a Trevanion-type null cipher. Then, you wrote a second program that generates a null cipher and conceals it within a language learner's vocabulary list. In the following practice projects, you can explore ways to make this list cipher more secure.

Further Reading

More details on Mary, Queen of Scots, and Sir John Trevanion can be found in *Mysterious Messages: A History of Codes and Ciphers* (The Penguin Group, 2009) by Gary Blackwood and *The Code Book: The Science of Secrecy from Ancient Egypt to Quantum Cryptography* (Anchor, 2000) by Simon Singh.

Practice Projects

Now that you're an expert on the null cipher, see if you can change the fate of Mary, Queen of Scots, and then sneak a look at Sir John's most secret correspondence.

Saving Mary

The best part of coding is thinking about problems and how to solve them. Let's revisit the sad case of Mary, Queen of Scots. Here's what we know:

- Mary was not allowed correspondence, so letters had to be smuggled in. This means that the traitorous Gilbert Gifford cannot be removed from the equation. Gifford was the only person Mary knew with the means to deliver her mail.
- Mary and her correspondents put too much faith in an insecure cipher and thus spoke too freely. With less confidence, they might have shown more forbearance.
- Mary's jailors, having an obvious cipher in their possession, assumed it contained incriminating material and kept working until they found it.

Gifford, the double agent, wasn't privy to the details of the ciphers Mary used. Now, assume Mary used a null cipher. If the correspondence was somewhat seditious—though not treasonously so—the message might have been overlooked by her captors. In the event a cursory examination was made, the use of a variable pattern might have sufficed to stymie the cryptanalysts.

As you have seen, it's easier to hide a null cipher in a list than in a letter. A list of families supporting Mary could serve the purpose. These could be known supporters or, in a Machiavellian twist, a mix of friends *and* enemies! This message wouldn't be openly seditious, but would be close enough so that a lack of encryption would suggest no form of encryption was being used at all.

For this practice project, write a program that embeds the message "Give your word and we rise" in a list of surnames. To hide the letters in the message, start at the second letter in the second name, move to the third

letter in the third name, and then keep alternating between second and third letters for the remaining words.

In addition to the unused first name, include “Stuart” and “Jacob” as null words early in the list to help hide the presence of the cipher. Don’t embed letters from the cipher in these null names and completely ignore them when choosing the letter position for the cipher in the following word; if the second letter was used in the word *before* the null name, use the third letter in the word *after* the null name. The null cipher would occupy the following bolded letters (the location of the null words is up to you, but don’t let them affect the pattern):

```
First Second Third STUART Fourth Fifth JACOB Sixth Seventh Eighth
```

The program can print the list either vertically or horizontally. The name list should be credibly introduced with a short message, but that message shouldn’t be part of the cipher.

The list of names can be downloaded from

<https://www.nostarch.com/impracticalpython/> as *supporters.txt* and loaded as a standard dictionary file. You can find a solution in the appendix and online as *save_Mary_practice.py*.

The Colchester Catch

Instead of some ale-sotted dimwit, *you* are left in charge of the prisoner John Trevanion when the following letter arrives at Colchester Castle:

Sir John: Odd and too hard, your lot. Still, we will band together and, like you, persevere. Who else could love their enemies, stand firm when all others fail, hate and despair? While we all can, let us feel hope. -R.T.

It seems clumsily phrased, even for the 17th century, and you decide to examine it more closely before passing it to your inmate.

Write a Python program that takes an input, *n*, and checks for and displays a null cipher based on the *n*th letter after the start of every *n*th

word. For example, an input of 2 would find the bolded letters in this message:

So, **th**e cold **te**a didn't **pl**ease the **ol**d finicky **w**oman.

You can download the text file of the message from

<https://nostarch.com/impracticalpython/> as *colchester_message.txt*. A

solution can be found in the appendix and online as

colchester_practice.py. Keep the text and Python files together in the same folder.